

Guia da apresentação

Q-Learning aplicado ao parkour

Roteiro de 15 minutos, fundamentos da modelagem, parâmetros do agente, resultados do treinamento e mapa das principais partes do código.

3.500	3.456	7,20%
EPISÓDIOS	ESTADOS	CHEGADAS NO TREINO

Treinamento analisado: frente_1 - 70 bots × 50 rodadas.

1. Mensagem central

O projeto modela um problema de controle sequencial como aprendizado por reforço tabular. O agente observa uma representação discreta da situação, escolhe entre quatro ações, recebe uma recompensa e atualiza uma tabela Q pela equação de Bellman.

Minecraft não é o assunto da apresentação. Ele é somente o ambiente 3D que fornece física, posição, velocidade e consequências reais das ações. O mapa do labirinto serve apenas como local onde os percursos de parkour foram construídos. Não apresentar BFS, DFS, A* ou resolução de labirintos.

Frase curta para abrir:

“Nosso problema não é encontrar a saída de um labirinto. É ensinar um agente a decidir quando andar, correr ou pular, aprendendo pelas consequências de cada ação.”

2. Divisão sugerida dos 15 minutos

Slides	Tema	Tempo	Pessoa sugerida
1–2	Problema de IA e formulação	1min40s	Integrante 1
3–4	Conversão do percurso e modelagem	3min05s	Integrante 2
5–6	Q-Learning, parâmetros e ciclo	2min50s	Integrante 2
7–8	Implementação e recompensa	2min25s	Integrante 3
9–10	Resultados e avaliação	2min35s	Integrante 4
11	Conclusão	0min45s	Integrante 4
—	Margem para troca de pessoas e pequenas pausas	1min40s	—

3. Roteiro slide a slide

Slide 1 — Q-Learning para controle de parkour — 0min30s

- Apresentar a pergunta: como aprender o momento de andar, correr ou pular sem programar uma regra para cada obstáculo?
- Dizer em uma frase que Minecraft fornece apenas o ambiente físico 3D.
- Antecipar: estado discreto, política epsilon-greedy, recompensa e atualização da tabela Q.

Slide 2 — Problema de decisão sequencial — 1min10s

- Cada ação muda a situação física seguinte; portanto, não é uma classificação isolada.
- Elementos do aprendizado por reforço:
 - agente: o bot;
 - ambiente: o mundo 3D;
 - estado: descrição compacta do que importa para decidir;
 - ação: andar, correr e variantes com pulo;
 - recompensa: sinal numérico do objetivo;
 - política: epsilon-greedy.
- Objetivo: maximizar a recompensa acumulada, não somente a recompensa do passo atual.

Slide 3 — Conversão do percurso 3D — 1min30s

Explicar o pipeline em três partes:

- 1 A ferramenta lê os arquivos regionais do mundo e identifica os blocos no corredor entre início e fim.
- 2 Cada bloco é reduzido a uma caixa de colisão com altura e largura. Blocos sem colisão são descartados.
- 3 O percurso é rotacionado e transladado para coordenadas locais: **lateral**, **altura** e **progresso**. Assim, caminhar para a meta sempre aumenta **progresso**, mesmo que no mundo real a pista esteja orientada em outro eixo.
- 4 A geometria local é salva em JSON. Durante o treino, o programa consulta esse mapa para saber onde há apoio, obstáculo e espaço para o corpo.
- 5 O estado olha as quatro células seguintes e produz um índice de **0 a 3455**.

Ponto importante: o JSON representa a geometria estática. A posição e a velocidade do agente continuam vindo do ambiente real durante o treino.

Slide 4 — Estado, ações e recompensa — 1min35s

O estado do cenário **pisso** combina:

- máscara de apoio nas quatro células seguintes: $2^4 = 16$ possibilidades;
- altura relativa do próximo apoio: **6** classes;
- posição dentro do bloco atual: **4** quartos;
- fase vertical: **3** classes — no chão, subindo ou descendo;
- velocidade de avanço: **3** faixas.

Quantidade total:

$16 \times 6 \times 4 \times 3 \times 3 = 3.456$ estados

As quatro ações permitidas são:

Índice	Ação
0	andar
1	correr
2	correr + pular
3	andar + pular

Cada ação é mantida por quatro ticks, aproximadamente 200 ms.

Slide 5 — Equação e parâmetros — 1min35s

Equação usada:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Como explicar os símbolos:

- **$Q(s,a)$** : valor atual de realizar a ação **a** no estado **s**;
- **r**: recompensa recebida;
- **$\max Q(s',a')$** : melhor valor conhecido no próximo estado;
- **α** : quanto a experiência nova corrige o valor antigo;
- **γ** : quanto valorizamos recompensas futuras.

Parâmetros do projeto:

Parâmetro	Valor	Interpretação
Taxa de aprendizado α	0,20	incorpora 20% do erro temporal em cada atualização
Desconto γ	0,97	mantém alta importância para consequências futuras
Exploração inicial ϵ	1,00	início totalmente exploratório
Exploração mínima	0,05	mantém 5% de exploração durante treino prolongado
Decaimento por episódio	0,9995	$\epsilon \leftarrow \max(0,05; \epsilon \times 0,9995)$
Exploração ao expandir ações	0,30	usada ao migrar uma tabela para um conjunto maior de ações

Na avaliação, $\epsilon = 0$: o agente usa somente a melhor ação aprendida.

Slide 6 — Ciclo do episódio — 1min15s

- 1 Observar o estado **s**.
- 2 Escolher a ação **a** pela política epsilon-greedy.
- 3 Executar a ação por quatro ticks.
- 4 Observar recompensa **r**, próximo estado **s'** e condição terminal.
- 5 Atualizar **$Q(s,a)$** .

- 6 Repetir até meta, queda, travamento ou limite de 80 decisões.
- 7 Ao fim do episódio, reduzir epsilon e registrar os resultados.

Slide 7 — Organização da implementação — 1min15s

- **q_learning.py**: algoritmo puro; não conhece jogo, mapa ou física.
- **estado.py**: transforma a observação em um índice inteiro.
- **recompensa.py**: transforma comportamento em um sinal numérico.
- **treinar.py**: organiza a transição (**s, a, r, s'**) ao longo do episódio.
- **ambiente_mc.py**: adaptador que executa ações e lê o ambiente real.

Mensagem principal: a fórmula de Bellman é curta. A maior decisão de IA foi escolher uma representação de estado e uma recompensa que contenham informação suficiente sem explodir o tamanho da tabela.

Slide 8 — Função de recompensa — 1min10s

Recompensa por decisão:

$$r = 1,0 \times \Delta\text{progresso} - 0,02$$

Ajustes:

- se o deslocamento for menor que **0,05**, soma **-0,05** por ficar parado;
- queda: **-10**;
- travamento: **-10**;
- meta: **+20**.

Nos percursos usados, o progresso só é confirmado depois de pousar em um apoio mapeado. Isso evita premiar um avanço no ar que termina em queda.

Também usamos progresso líquido. Se o agente avança e volta, o ganho é cancelado. Isso reduz uma forma de *reward hacking* em que ele poderia oscilar sem se aproximar da meta.

Slide 9 — Resultados do treinamento salvo — 1min20s

O treinamento mais recente do percurso **frente_1** usou 70 bots e 50 rodadas por bot:

$$70 \times 50 = 3.500 \text{ episódios}$$

O CSV foi limpo e contém somente esses 3.500 episódios, numerados de 1 a 3.500.

Resultados do histórico válido:

Janela do histórico	Episódios	Chegadas	Taxa	Retorno médio	Progresso válido médio
Primeiros 1.000 registros	1.000	60	6,00%	4,34	26,3%
Últimos 1.000 registros	1.000	104	10,40%	9,84	34,3%
Treino completo	3.500	252	7,20%	5,76	28,3%

Interpretação: a melhora entre o começo e o fim do histórico é evidência de aprendizado durante o treino. Entretanto, as ações ainda incluíam exploração epsilon-greedy; portanto, **10,40%** não é a taxa final da política determinística.

Arquivos salvos:

- modelo: **resultados/modelos/q_learning_labirinto_parkours_frente_1_acoes_v2_estado_v2_recompensa_v2.json**;
- histórico: **resultados/modelos/q_learning_labirinto_parkours_frente_1_acoes_v2_estado_v2_recompensa_v2_resultado.csv**.

Slide 10 — Treino versus avaliação — 1min15s

O histórico limpo mostra:

- epsilon **0,05** em todos os registros;
- retorno médio de **4,34** nos primeiros 1.000 e **9,84** nos últimos 1.000;
- progresso válido médio de **26,3%** nos primeiros 1.000 e **34,3%** nos últimos 1.000.

Ainda não existem linhas com **fase = avaliacao** no CSV. Todas as 3.500 linhas são de treino. A avaliação adequada agora é:

- 1 Carregar o modelo salvo.
- 2 Desligar exploração e aprendizado — o comando **parkour rodar** já usa $\epsilon = 0$ e **aprender = False**.
- 3 Repetir várias tentativas independentes.
- 4 Medir taxa de chegada, recompensa acumulada e passos até a meta.
- 5 Relatar separadamente as métricas de treino e de avaliação.

Slide 11 — Conclusão — 0min45s

- Representação: o ambiente 3D foi transformado em 3.456 estados significativos.
- Aprendizado: epsilon-greedy coleta experiências e Bellman atualiza os valores.
- Objetivo: a recompensa favorece progresso, rapidez e chegada, penalizando falhas.
- Próximo passo: carregar o modelo salvo e avaliar a política sem exploração.

Frase final:

“A principal contribuição não é uma regra de pulo, mas uma representação que permite ao agente aprender a regra por experiência.”

4. O que precisa ser entendido sobre a conversão

4.1 Leitura do mapa

O leitor abre os arquivos **.mca** do mundo e decodifica o formato NBT. A ferramenta de mapeamento consulta somente a caixa tridimensional ao redor do corredor definido por início e fim.

4.2 Simplificação geométrica

O algoritmo não precisa conhecer o nome visual de cada bloco. Ele precisa saber se o corpo consegue apoiar, passar ou colidir. Por isso, cada bloco é convertido em:

```
[posição lateral, altura da base, altura da colisão, largura da colisão]
```

Exemplos: bloco cheio = altura 1 e largura 1; laje = altura 0,5; elementos atravessáveis = altura e largura 0.

4.3 Sistema local de coordenadas

O início e o fim determinam um vetor “para frente”. Uma rotação de 90 graus e, quando necessário, uma reflexão tornam todas as pistas equivalentes:

```
coordenadas do mundo (x, y, z)
    ↓
coordenadas da IA (lateral, altura, progresso)
```

No sistema local, a largada é aproximadamente **progresso = 0** e a meta fica em **progresso = comprimento**. As pistas utilizadas têm 53 unidades de comprimento.

4.4 JSON intermediário

O JSON exportado registra mundo de origem, início, fim, direção, limites, pista detectada, nomes de blocos e caixas sólidas por posição de progresso. Ao carregar, o código valida se o JSON realmente corresponde ao cenário solicitado.

4.5 Estado usado pelo Q-Learning

Para cada decisão, o agente não percorre o JSON inteiro. Ele consulta somente o que é necessário: apoios nas quatro células seguintes, altura do primeiro apoio, posição fracionária, fase vertical e velocidade. Esses componentes são combinados por multiplicações sucessivas para gerar uma linha única da tabela Q.

5. Perguntas prováveis do professor

Por que Q-Learning tabular?

Porque o espaço foi discretizado para 3.456 estados e há somente quatro ações. Isso permite uma política interpretável, implementação curta e ligação direta com os conceitos introdutórios de Bellman, exploração e exploração versus aproveitamento.

O estado satisfaz a propriedade de Markov?

É uma aproximação. Incluímos geometria à frente, posição dentro do bloco, fase vertical e faixa de velocidade porque essas variáveis concentram a informação necessária para a próxima decisão. Ainda há perda de informação contínua, o que é uma limitação da discretização.

Por que não usar somente “há buraco ou não”?

Porque situações visualmente parecidas podem exigir ações diferentes. Um apoio pode estar na mesma altura, abaixo, acima mas alcançável, ou alto demais. A fase do salto e a velocidade também alteram a ação adequada.

Por que $\gamma = 0,97$?

Porque chegar à meta depende de várias decisões. Um desconto alto propaga o valor da chegada para estados anteriores, sem tornar presente e futuro exatamente equivalentes.

Por que $\alpha = 0,20$?

É uma atualização moderada: cada experiência influencia a estimativa, mas não apaga todo o histórico anterior. O valor é um hiperparâmetro e deve ser comparado experimentalmente em trabalhos futuros.

O que o decaimento de epsilon significa?

No início, com $\epsilon = 1$, o agente explora ações aleatórias. Após cada episódio, epsilon é multiplicado por **0,9995** até o piso **0,05**. Na avaliação, epsilon é zero.

Existe simulador ou treino fora do jogo?

Não. O JSON é uma representação estática da geometria usada para formar o estado. As ações, a posição, a velocidade e a física durante o treino vêm do ambiente real.

Por que usar progresso líquido?

Para impedir que o agente ganhe recompensa indo e voltando. O deslocamento negativo cancela o positivo, e ficar quase parado ainda recebe penalidade.

Os resultados provam que o agente aprendeu?

Eles mostram evidência de aprendizado durante o treino: a taxa de chegada passou de 6,00% nos primeiros 1.000 registros para 10,40% nos últimos 1.000, enquanto retorno e progresso médio também aumentaram. Ainda falta medir a política separadamente com epsilon zero para estimar seu desempenho sem ações exploratórias.

Quantos episódios entram nos resultados apresentados?

Somente os 3.500 episódios do CSV limpo: 70 bots × 50 rodadas. Os treinamentos anteriores foram removidos e não entram nas taxas, médias ou comparações apresentadas.

Por que quatro ticks por ação?

Decidir a cada tick quadruplicaria aproximadamente o número de decisões. Quatro ticks equivalem a cerca de 200 ms e ainda são bem menores que a duração aproximada de um salto.

6. Localizações importantes do código

As linhas abaixo correspondem ao estado atual do repositório.

Assunto	Localização	O que mostrar
Parâmetros principais	config/parkour.json:21-48	ticks, limite padrão, recompensa, α , γ e epsilon
Cenário dos três percursos	config/cenarios/labirinto_parkours.json:8-59	mapas, início/fim, modo piso , quatro ações e 80 passos
Transformação de coordenadas	src/parkour/coordenadas.py:18-122	criação do eixo local e conversões mundo ↔ local
Leitura/exportação do percurso	tools/mapear_percurso.py:58-166	leitura da caixa, transformação dos blocos e JSON final
Leitor dos arquivos do mundo	tools/nbt.py:1-170	decodificação NBT e acesso aos chunks .mca
Caixas de colisão	tools/blocos.py:1-122	altura/largura e descarte de blocos atravessáveis
Carregamento e validação do JSON	src/parkour/percurso.py:293-335	confere mundo, início, fim e cria eixo local 0 → meta
Quantidade de estados	src/parkour/estado.py:94-104	fórmula que retorna 3.456 estados no modo piso
Codificação do estado	src/parkour/estado.py:164-206	quatro células, posição, fase vertical e velocidade
Classes de altura do apoio	src/parkour/estado.py:208-229	seis categorias físicas do próximo apoio
Catálogo de ações	src/parkour/acoes.py:32-61	índices e combinações de controles
Parâmetros no agente	src/parkour/q_learning.py:36-65	inicialização de α , γ , epsilon e tabela
Política epsilon-greedy	src/parkour/q_learning.py:70-86	exploração versus melhor ação conhecida
Atualização de Bellman	src/parkour/q_learning.py:88-109	cálculo do alvo, erro e correção de Q
Decaimento de epsilon	src/parkour/q_learning.py:111-124	redução por episódio e avaliação com epsilon zero
Diagnóstico do modelo	src/parkour/q_learning.py:197-204	epsilon, estados visitados e cobertura
Função de recompensa	src/parkour/recompensa.py:32-60	progresso, custo por passo e sinais terminais
Execução da ação e término	src/parkour/ambiente_mc.py:148-218	quatro ticks, pouso, queda, meta, tempo e recompensa
Progresso normalizado no CSV	src/parkour/ambiente_mc.py:256-280	progresso bruto, progresso válido e chegada
Laço de um episódio	src/parkour/treinar.py:54-94	sequência escolher → agir → aprender
Registro de resultados	src/parkour/treinar.py:97-125	colunas e escrita do CSV
Treinamento paralelo	src/parkour/main.py:363-439	rodadas por bot, numeração, salvamento e histórico
Modelo Q salvo	resultados/modelos/q_learning_labirinto_parkours_frente_1_acoes_v2_estado_v2_recompensa_v2.json	tabela, visitas, epsilon e metadados
Histórico válido	resultados/modelos/q_learning_labirinto_parkours_frente_1_acoes_v2_estado_v2_recompensa_v2_resultado.csv	3.500 registros do treino mais recente

7. Números para memorizar

Estados: 3.456
 Ações: 4
 α : 0,20
 γ : 0,97
 ϵ inicial: 1,00
 ϵ mínimo: 0,05
 Decaimento de ϵ : 0,9995 por episódio
 Ticks por ação: 4 $\approx$ 200 ms
 Máximo por episódio: 80 decisões neste cenário
 Meta: +20
 Queda/travamento: -10
 Custo por decisão: -0,02
 Parado: -0,05 adicional
 Testes: 38/38
 Percursos: 3
 Comprimento: 53 unidades cada
 Histórico válido: 3.500 episódios
 Bots: 70
 Rodadas por bot: 50
 Chegadas no treino: 252 — 7,20%
 Primeiros 1.000: 6,00% de chegada
 Últimos 1.000: 10,40% de chegada
 Retorno médio: 4,34 $\rightarrow$ 9,84
 Progresso válido: 26,3% $\rightarrow$ 34,3%
 Epsilon no histórico: 0,05

8. Afirmações que devem ser evitadas

- Não dizer que o projeto resolve o labirinto.
- Não explicar BFS, DFS ou A*.
- Não dizer que há rede neural, DQN ou aprendizado profundo.
- Não dizer que o agente convergiu; os dados mostram evolução, não prova matemática de convergência.
- Não apresentar **10,40%** como taxa final da política: é a taxa nos últimos 1.000 episódios de treino com exploração.
- Não misturar os treinamentos antigos com o histórico atual: as métricas usam somente as 3.500 linhas do CSV limpo.
- Não inventar taxa de avaliação: ainda não existem registros com **fase = avaliacao**.
- Não dizer que o treino usa um simulador próprio; a geometria é exportada, mas a dinâmica vem do ambiente real.
- Não gastar tempo explicando regras ou itens do jogo. Traduzir tudo para estado, ação, transição e recompensa.

9. Checklist antes de apresentar

- Cada integrante sabe explicar a diferença entre estado contínuo observado e estado discreto da tabela.
- Quem apresentar o slide 3 sabe desenhar o pipeline **mundo** → **colisões** → **eixo local** → **estado**.
- Quem apresentar o slide 5 sabe ler a equação sem decorar cada símbolo isoladamente.
- Quem apresentar o slide 8 sabe explicar progresso líquido e *reward hacking*.
- O grupo distingue **38/38** testes de corretude, 252 chegadas durante treino e a futura taxa de avaliação.
- O grupo memoriza a configuração correta: 70 bots × 50 rodadas = 3.500 episódios.
- O grupo usa somente os 3.500 registros do histórico limpo ao citar resultados.
- O grupo ensaia uma vez com cronômetro e preserva pelo menos um minuto para perguntas.